Universidad Tecnológica de El Salvador

FACULTAD DE INFORMÁTICA Y CIENCIAS APLICADAS

ESCUELA DE INFORMÁTICA

BASE DE DATOS II

DOCENTE:

ING. JOSE ORLANDO GIRON BARRERA.

SECCIÓN:

03.

TEMA DE INVESTIGACIÓN:

BASES DE DATOS NOSQL CLAVE-VALOR Y CACHING (REDIS / MEMCACHED).

INTEGRANTE:

Nº	CARNET	APELLIDOS	NOMBRES	PORCENTAJE DE TRABAJO
1	25-0005-2020	Chávez Alfaro	Josué Adalberto	100%

SAN SALVADOR, 15 DE NOVIEMBRE DE 2026.

2. Índice de contenido

1. Portada 1

2. Índice de contenido 2

3. Objetivos del proyecto 4

3.1 Objetivo general 4

3.2 Objetivos específicos 4

4. Definición e introducción del proyecto y del motor escogido 5

4.1 Problema que se resuelve 5

4.2 Bases de datos clave-valor 5

4.3 Redis 5

4.4 Memcached 6

4.5 Comparación y motor escogido 6

4.6 Nota sobre licencias 7

4.7 Arquitectura de la solución 7

5. Organización y arquitectura de la información 8

5.1 Diccionario de datos de Redis 8

5.2 Tipos de datos seleccionados 9

5.3 Datos de ejemplo 9

6. Diagramas y modelos de datos 10

6.1 Modelo conceptual 10

6.2 Modelo lógico 10

6.3 Modelo físico 11

6.4 Arquitectura y flujo 12

7. Diseño de interfaz y prototipo de interacción 13

7.1 Herramientas del prototipo 13

7.2 Capturas del prototipo 14

7.3 Página web de la tienda 15

8. Pruebas, resultados y conclusiones 17

8.1 Plan de pruebas 17

8.2 Resultados esperados de los scripts 19

8.3 Conclusiones 20
8.4 Referencias bibliográficas 21
9. Código fuente y scripts 22
9.1 Levantar Redis (Docker) 22
9.2 Ejecutar los scripts 22
9.3 Datos: 01_datos.redis 22
9.4 Consultas y DML: 02_consultas_y_dml.redis 23
9.5 Transacciones: 03_transacciones.redis 24
9.6 Funciones almacenadas: 04_funciones.redis 24
9.7 Consultas complejas: 05_consultas_complejas.redis 25
9.8 Trigger equivalente: notificaciones de expiración 26
9.9 Prueba de caché en Python: prueba_cache.py 27
9.10 Seguridad 28
9.11 Página web de la tienda: app.py 28
10. Publicación en versionadores 32

3. Objetivos del proyecto

3.1 Objetivo general

Diseñar e implementar una solución de almacenamiento en memoria con Redis para una tienda en línea, que reduzca los tiempos de respuesta de las lecturas repetidas y resuelva problemas reales de carritos, sesiones, rankings y control de stock, aplicando estándares de modelado, optimización y seguridad.

3.2 Objetivos específicos

- Definir el esquema de claves de Redis (nombres, tipos de datos y tiempos de expiración) para los datos de la tienda.

- Implementar scripts de consulta y manipulación (DML) con comandos de Redis, una transacción con WATCH/MULTI/EXEC, funciones almacenadas en Lua que protejan la integridad de los datos, consultas complejas con Sets y Sorted Sets, y un trigger equivalente basado en notificaciones de claves.

- Desarrollar un prototipo de pruebas (Redis Insight y un script en Python) que demuestre la interacción con los datos y mida la mejora con y sin caché.

- Comparar Redis con Memcached y justificar la elección del motor.

- Desarrollar una página web sencilla de la tienda que permita probar el carrito, las compras, el ranking y la caché desde el navegador.

4. Definición e introducción del proyecto y del motor escogido

4.1 Problema que se resuelve

Una tienda en línea recibe muchas lecturas repetidas (catálogo, detalle de producto, ranking de ventas) y pocas escrituras (compras). Si cada visita consulta directamente la base de datos principal, esta se satura en horas pico y los tiempos de respuesta aumentan. Además, la tienda necesita manejar carritos, sesiones temporales y el control de stock cuando varios clientes compran a la vez.

En este proyecto la base de datos principal se simula con una función de Python que tarda 200 ms por consulta, para concentrar el trabajo en Redis, que es el tema escogido.

4.2 Bases de datos clave-valor

Una base de datos clave-valor guarda la información como pares clave-valor y casi siempre la recupera por la clave exacta. No exige un esquema fijo ni usa joins, lo que la hace muy rápida y fácil de escalar. Motores como Redis y Memcached guardan los datos principalmente en memoria RAM, por lo que responden en microsegundos o pocos milisegundos.

4.3 Redis

Redis (Remote Dictionary Server) es un almacén de estructuras de datos en memoria que se usa como base de datos, caché y broker de mensajes. Sus características principales son:

- **Estructuras de datos ricas:** strings, hashes, listas, conjuntos, conjuntos ordenados, streams y bitmaps.
- **Persistencia opcional:** snapshots RDB y registro de operaciones AOF.
- **Expiración automática** de claves mediante TTL y políticas de expulsión cuando se llena la memoria.
- **Transacciones** con WATCH/MULTI/EXEC y **funciones en Lua** (Redis Functions) que se ejecutan de forma atómica.

- **Notificaciones de eventos de claves** (keyspace notifications), que avisan cuando una clave expira o cambia.
- **Replicación, Sentinel y Cluster** para alta disponibilidad y escalado.

4.4 Memcached

Memcached es un sistema de caché distribuida en memoria, sencillo y muy eficiente para guardar objetos pequeños como strings. Es multihilo, pero no ofrece estructuras de datos ricas, persistencia, replicación nativa ni scripting.

4.5 Comparación y motor escogido

Característica	Redis	Memcached
Tipos de datos	Strings, hashes, listas, sets, sorted sets, streams, bitmaps	Solo strings/blobs
Persistencia	Sí (RDB y AOF)	No
Alta disponibilidad	Réplicas, Sentinel, Cluster	No nativa; la distribución la hace el cliente
Funciones y transacciones	Funciones Lua, WATCH/MULTI/EXEC	No
Eventos (equivalente a triggers)	Notificaciones de claves y Pub/Sub	No
Expiración	TTL por clave y políticas de expulsión	TTL por clave y expulsión LRU
Uso ideal	Caché con estructuras, sesiones, rankings, contadores	Caché simple de objetos

Motor escogido: Redis.

La tienda necesita más que un caché de strings: carritos (hashes), ranking de ventas (sorted sets), sesiones con expiración y control de stock con funciones Lua. Memcached no cubre estos casos sin lógica adicional en la aplicación.

4.6 Nota sobre licencias

En marzo de 2024 Redis cambió su licencia de BSD a un modelo dual SSPL/RSALv2 desde la versión 7.4. Como respuesta nació Valkey, un fork de Redis 7.2 bajo la Linux Foundation y licencia BSD. Con Redis 8 se añadió AGPLv3 como opción de licencia. Valkey mantiene compatibilidad de protocolo con Redis, por lo que este diseño podría migrarse a Valkey con pocos cambios; conviene verificar la compatibilidad de comandos concretos antes de hacerlo.

4.7 Arquitectura de la solución

La solución usa el patrón **cache-aside:** el cliente consulta primero Redis; si la clave existe (HIT) responde de inmediato; si no existe (MISS), se consulta la base de datos, se guarda el resultado en Redis con un TTL y se responde. Cuando un dato cambia en la base de datos, se borra su clave de Redis para que la siguiente lectura traiga el valor nuevo (ver Figura 3).

5. Organización y arquitectura de la información

5.1 Diccionario de datos de Redis

Clave	Tipo	TTL	Contenido	Para qué sirve
`producto:{id}`	Hash	600 s (10 min)	nombre, precio	Caché de lecturas repetidas; cada campo se lee por separado
`stock:{id}`	String (entero)	Sin TTL	unidades disponibles	Inventario; `DECRBY` es atómico y el dato no debe expirar
`carrito:{usuario_id}`	Hash	7 días	campo = id del producto, valor = cantidad	Carrito; se modifica con `HINCRBY`
`ranking:ventas`	Sorted Set	Sin TTL	miembro = id del producto, puntaje = unidades vendidas	Top N en tiempo real con `ZREVRANGE`
`categoria:{nombre}`	Set	Sin TTL	ids de los productos de la categoría	Índice secundario: buscar por categoría y filtrar rankings
`sesion:{token}`	String	30 min (1800 s)	id del usuario	Sesión temporal que expira sola

5.2 Tipos de datos seleccionados

- **Hash** para objetos con varios campos que se leen o modifican por separado (producto, carrito).
- **String con entero** para el stock, porque DECRBY lo descuenta de forma atómica.
- **Sorted Set** para el ranking, porque mantiene los productos ordenados por puntaje.
- **Set** para agrupar los productos por categoría y combinarlos con otras estructuras (índice secundario).
- **TTL** en los datos que pueden recrearse (caché de producto, sesión, carrito); sin TTL en los datos que no deben perderse (stock y ranking).
- **Convención de nombres:** entidad:identificador, en minúsculas y con dos puntos como separador.

5.3 Datos de ejemplo

id	nombre	precio	stock inicial	categoría
1	Auriculares inalámbricos	29.99	50	tecnologia
2	Teclado mecanico	45.50	30	tecnologia
3	Lampara LED	12.00	100	hogar

6. Diagramas y modelos de datos

6.1 Modelo conceptual

Las entidades son Usuario, Sesión, Carrito, Producto, Stock, Ranking y Categoría. Un usuario tiene un carrito y puede abrir varias sesiones; un carrito contiene muchos productos y un producto puede estar en muchos carritos (N:M); cada producto tiene un stock, una posición en el ranking de ventas y pertenece a una categoría.

Modelo conceptual de la tienda

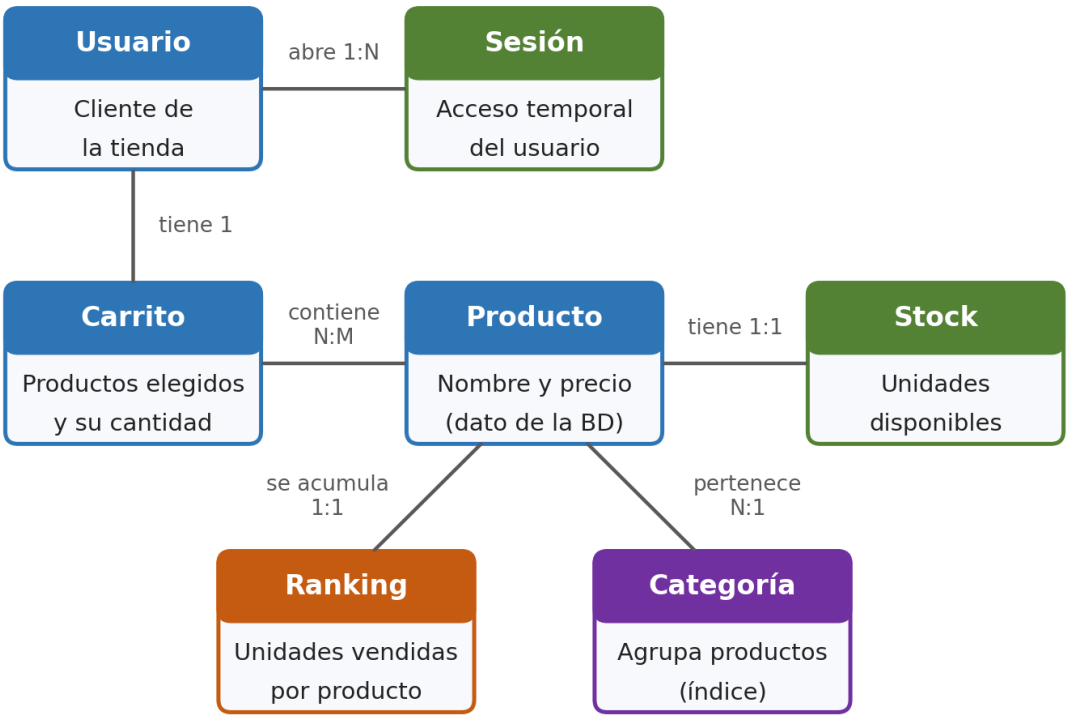


Figura 1. Modelo conceptual de la tienda

6.2 Modelo lógico

Redis no usa tablas ni formas normales. El modelo se diseña a partir de las consultas que la tienda necesita, casi todas por clave exacta, por lo que los datos se guardan listos para leer y sin joins. La relación N:M entre carrito y producto se resuelve con un Hash

cuyo campo es el id del producto. La fuente de verdad del catálogo es la base de datos principal; producto:{id} es solo una copia temporal.

Índices. En Redis la propia clave es el índice primario: leer producto:{id} por clave exacta es una operación directa, sin recorrer datos. Los índices secundarios se construyen con estructuras: el Set categoria:{nombre} permite buscar productos por categoría y el Sorted Set ranking:ventas es un índice ordenado por unidades vendidas.

Integridad referencial. Redis no tiene claves foráneas. Esa integridad se garantiza dentro de las funciones Lua, que verifican que el producto exista antes de venderlo o agregarlo al carrito (devuelven -1 si no existe) y que descuentan el stock y actualizan el ranking en un solo paso atómico.

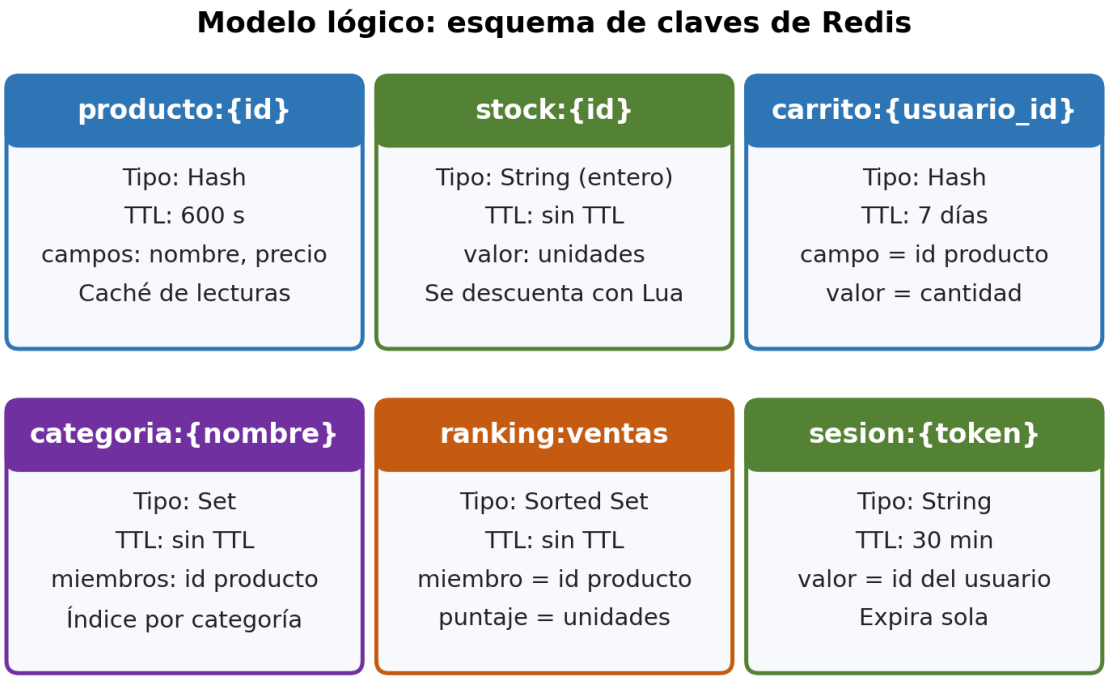


Figura 2. Modelo lógico: esquema de claves de Redis

6.3 Modelo físico

El modelo físico son los scripts de la carpeta `scripts/` (secciones 9.3 a 9.7) y la configuración con la que se levanta Redis (sección 9.1):

- Persistencia AOF (`--appendonly yes`, `--appendfsync everysec`) para no perder datos si Redis se reinicia.
- Límite de memoria de 256 MB con la política `volatile-lru`, que solo expulsa claves con TTL (caché, carritos y sesiones) y protege el stock y el ranking.
- Notificaciones de expiración de claves (`--notify-keyspace-events Ex`), que se usan como trigger (sección 9.8).

6.4 Arquitectura y flujo

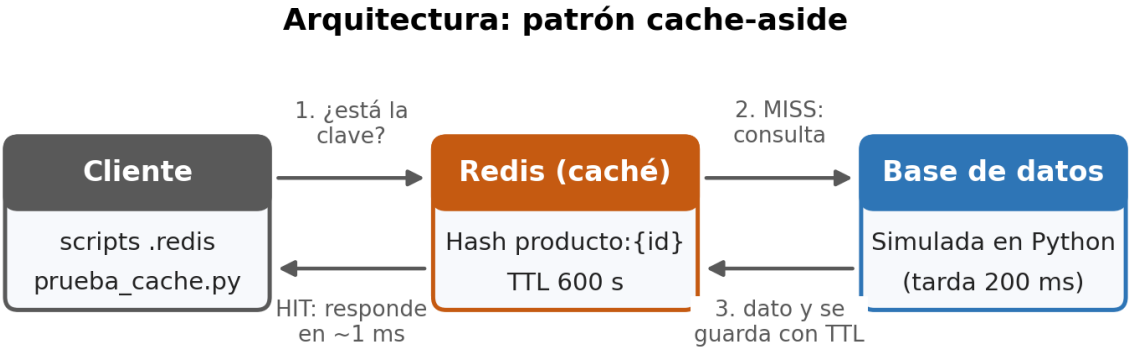


Figura 3. Arquitectura de la solución (cache-aside)

Flujo de lectura: el cliente consulta producto:{id}; si hay HIT responde Redis, y si hay MISS se consulta la base de datos, se guarda el resultado con TTL de 600 s y se responde. **Flujo de escritura:** al cambiar un dato en la base de datos se borra la clave con DEL para evitar datos obsoletos.

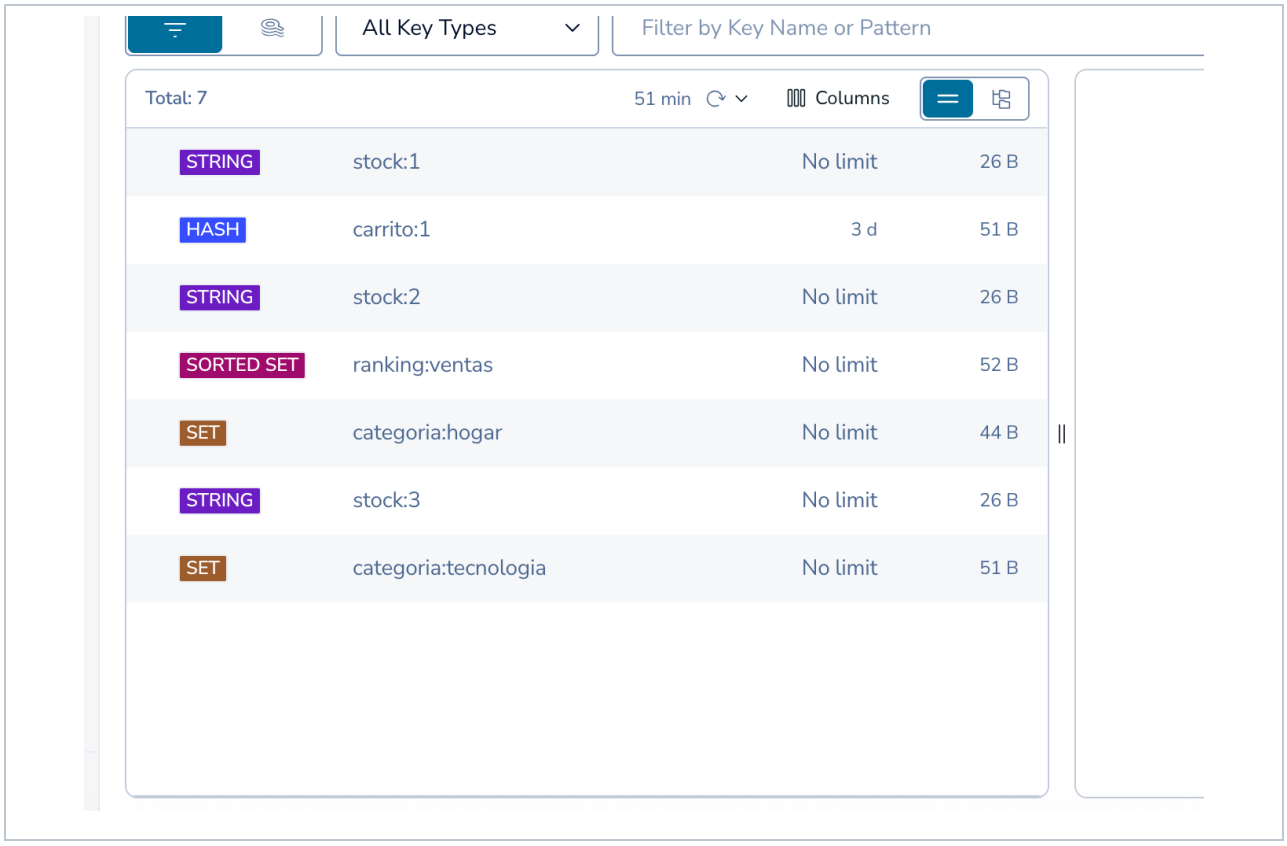
7. Diseño de interfaz y prototipo de interacción

7.1 Herramientas del prototipo

Herramienta	Para qué se usa
`Redis Insight`	Interfaz gráfica gratuita para ver las claves, sus tipos, sus valores y su TTL en tiempo real
`redis-cli`	Ejecutar los scripts de la carpeta scripts/ y las pruebas manuales
`prueba_cache.py`	Script en Python que mide el tiempo de respuesta con y sin caché
`Página web (app.py)`	Interfaz de la tienda hecha con Flask: muestra si cada producto sale de la caché (HIT) o de la base de datos (MISS), el carrito, las compras y el ranking

7.2 Capturas del prototipo

Prueba 1: Redis Insight mostrando las claves, sus tipos y su TTL:



Prueba 2: Salida de prueba_cache.py:

```
josuealfaro@MacBook-Pro-de-Josue tienda-redis % cd ~/Desktop/tienda-redis
python3 -m venv .venv
source .venv/bin/activate
pip install redis
python prueba_cache.py
Collecting redis
  Downloading redis-7.0.1-py3-none-any.whl (339 kB)
     |████████████████████████████████| 339 kB 1.1 MB/s
Collecting async-timeout>=4.0.3
  Downloading async_timeout-5.0.1-py3-none-any.whl (6.2 kB)
Installing collected packages: async-timeout, redis
Successfully installed async-timeout-5.0.1 redis-7.0.1
WARNING: You are using pip version 21.2.4; however, version 26.0.1 is available.
You should consider upgrading via the '/Users/josuealfaro/Desktop/tienda-redis/.venv/bin/python3
SIN cache: promedio de 10 consultas = 201.5 ms
CON cache: primera consulta (MISS) = 207.7 ms
CON cache: promedio de las 9 siguientes (HIT) = 0.87 ms
La cache fue 231 veces mas rapida
TTL restante de producto:1 = 600 segundos
(.venv) josuealfaro@MacBook-Pro-de-Josue tienda-redis %
```

Prueba 3: Ejecución de los scripts en la terminal:

```
josuealfaro@MacBook-Pro-de-Josue ~ % cd ~/Desktop/tienda-redis
cat scripts/*.redis | docker exec -i redis-tienda redis-cli > salida.txt
grep -c ERR salida.txt
tail -n 5 salida.txt
0
3
4
48
29
106
```

7.3 Página web de la tienda

La página `app.py` se conecta a Redis y simula una tienda con tres productos. Para abrirla, instala Flask con `pip install flask`, ejecuta `python app.py` y entra a http://localhost:5001 en el navegador. Funciona así:

- Cada producto muestra de dónde salió el dato y cuánto tardó: la primera lectura es MISS (va a la base de datos simulada, unos 200 ms) y las siguientes son HIT (salen de Redis).
- **Agregar al carrito** llama a la función `agregar_carrito`, que verifica que el producto exista.
- **Comprar 1** llama a la función `comprar`, que descuenta el stock solo si hay unidades y actualiza el ranking.
- **Borrar caché** elimina las claves `producto:*` para volver a ver un MISS.
- A la derecha se muestran el carrito del usuario 1 y los productos más vendidos.

Prueba 4: Página web de la tienda (`app.py`) con los productos, el carrito y los más vendidos:



8. Pruebas, resultados y conclusiones

8.1 Plan de pruebas

#	Prueba	Cómo hacerla	Qué anotar	Resultado
1	Con caché vs sin caché	Ejecutar `python prueba_cache.py`	Promedio sin caché, primera consulta (MISS) y promedio con caché (HIT)	**SIN cache:** promedio de 10 consultas = 201.5 ms **CON cache:** primera consulta (MISS) = 207.7 ms **CON cache:** promedio de las 9 siguientes (HIT) = 0.87 ms La cache fue 231 veces mas rapida
2	Expiración por TTL y trigger	En `redis-cli`: `EXPIRE producto:2 10`, esperar 10 s y ejecutar `EXISTS producto:2`. Con la suscripción de la sección 9.8 abierta, mirar el aviso	Que `EXISTS` devuelva 0 y que la suscripción muestre el evento de expiración	Se asignó un TTL de 10 s a producto:2. Al vencer, la suscripción a __keyevent@0__:expired mostró el evento con la clave producto:2, lo que demuestra el trigger por expiración. EXISTS producto:2 devolvió 0.

#	Prueba	Cómo hacerla	Qué anotar	Resultado
3	Persistencia	Ejecutar los scripts, correr `docker restart redis-tienda` y luego `ZREVRANGE ranking:ventas 0 4 WITHSCORES`	Que el ranking y el stock sigan igual después del reinicio (AOF)	Antes de reiniciar, ZREVRANGE ranking:ventas 0 4 WITHSCORES devolvió el producto 3 con puntaje 4, el producto 1 con puntaje 2 y el producto 2 con puntaje 1. Tras docker restart redis-tienda, el resultado fue idéntico, por lo que los datos persistieron gracias a AOF.

8.2 Resultados esperados de los scripts

Al ejecutar los 5 scripts en orden, las funciones de `04_funciones.redis` devolvieron lo siguiente:

Comando	Resultado esperado
`FCALL comprar 2 stock:1 ranking:ventas 2 1`	`1`: compra de 2 unidades
`FCALL comprar 2 stock:1 ranking:ventas 500 1`	`0`: stock insuficiente, no cambia nada
`FCALL comprar 2 stock:3 ranking:ventas 4 3`	`1`: compra de 4 unidades
`FCALL comprar 2 stock:99 ranking:ventas 1 99`	`-1`: el producto no existe
`FCALL agregar_carrito 2 stock:2 carrito:1 2 1`	`1`: producto agregado al carrito
`FCALL agregar_carrito 2 stock:99 carrito:1 99 1`	`-1`: el producto no existe, no se agrega

Al terminar 05_consultas_complejas.redis nos dio:

- GET stock:1, GET stock:2 y GET stock:3 devuelven 48, 29 y 106.
- ZREVRANGE ranking:ventas 0 2 WITHSCORES devuelve el producto 3 con puntaje 4, el producto 1 con puntaje 2 y el producto 2 con puntaje 1.
- ZREVRANK ranking:ventas 1 devuelve 1 (el producto 1 es el segundo del ranking, contando desde 0).
- ranking:tecnologia contiene el producto 1 (puntaje 2) y el producto 2 (puntaje 1); ranking:hogar contiene solo el producto 3 (puntaje 4).
- HGETALL carrito:1 (última línea de 04_funciones.redis) devuelve el producto 1 con cantidad 3 y el producto 2 con cantidad 1.

8.3 Conclusiones

- Una capa de caché en memoria reduce la latencia de las lecturas repetidas y descarga a la base de datos principal. En la prueba 1, consultar la base de datos simulada tardó en promedio 201.5 ms; con caché, la primera lectura (MISS) tardó 207.7 ms y las 9 siguientes (HIT) promediaron 0.87 ms, es decir, la caché fue unas 231 veces más rápida.

- **Trigger por expiración:** al vencer el TTL de 10 s de producto:2, la suscripción recibió el evento y EXISTS devolvió 0. Como es publicar/suscribir, si nadie escucha el aviso se pierde.

- **Persistencia AOF:** el ranking fue idéntico antes y después de docker restart. Con appendfsync everysec, ante una caída abrupta podría perderse cerca de un segundo de escrituras.

- Redis es más adecuado que Memcached cuando se necesitan estructuras de datos (Hash, Sorted Set), expiración flexible y persistencia, como en el carrito y el ranking.

- MULTI/EXEC ejecuta los comandos juntos y WATCH evita conflictos entre clientes, pero la transacción no valida condiciones ni revierte; para descontar stock solo si hay unidades y verificar que el producto exista se usan funciones Lua, que se ejecutan de forma atómica.

- Redis no tiene claves foráneas ni triggers como un motor relacional; sus equivalentes son las validaciones dentro de las funciones Lua, los Sets y Sorted Sets como índices y las notificaciones de eventos de claves.

- **Redis complementa a la base de datos principal y no la sustituye:** el TTL limita el tiempo que un dato de caché puede quedar obsoleto y la política volatile-lru protege el stock y el ranking.

- **Alcance:** la base de datos principal se simuló con una función de Python; en un sistema real sería un motor relacional y la caché se invalidaría al actualizar cada dato.

- **Página web:** la página en Flask permitió probar desde el navegador el patrón cache-aside, el carrito y las compras, que rechazan los productos inexistentes o sin stock.

8.4 Referencias bibliográficas

Lorusso, L. (2025, 22 de abril). *Forking ahead: A year of Valkey*. Linux Foundation. https://www.linuxfoundation.org/blog/a-year-of-valkey

Memcached. (s. f.). *Memcached: A distributed memory object caching system*. https://memcached.org

Redis Ltd. (s. f.-a). *Redis Insight*. https://redis.io/insight/

Redis Ltd. (s. f.-b). *Welcome to Redis Docs*. https://redis.io/docs/latest/

Valkey. (s. f.). *Valkey documentation*. https://valkey.io/docs/

9. Código fuente y scripts

9.1 Levantar Redis (Docker)

```
docker run -d --name redis-tienda -p 6379:6379 -v redis-data:/data redis:8 redis-server --appendonly yes --appendfsync everysec --maxmemory 256mb --maxmemory-policy volatile-lru --notify-keyspace-events Ex
```

9.2 Ejecutar los scripts

```
cat scripts/*.redis | docker exec -i redis-tienda redis-cli
```

PowerShell:

```
Get-Content scripts\*.redis | docker exec -i redis-tienda redis-cli
```

9.3 Datos: 01_datos.redis

`FLUSHDB` borra la base de pruebas para empezar siempre desde cero. Cada `HSET`, `SET` y `SADD` crea una clave con el tipo del diccionario de datos, y `EXPIRE` o `EX` les asignan su `TTL`.

```
FLUSHDB
HSET producto:1 nombre "Auriculares inalambricos" precio 29.99
HSET producto:2 nombre "Teclado mecanico" precio 45.50
HSET producto:3 nombre "Lampara LED" precio 12.00
EXPIRE producto:1 600
EXPIRE producto:2 600
EXPIRE producto:3 600
SET stock:1 50
SET stock:2 30
SET stock:3 100
SADD categoria:tecnologia 1 2
SADD categoria:hogar 3
```

```
HINCRBY carrito:1 1 2
HINCRBY carrito:1 3 1
EXPIRE carrito:1 604800
SET sesion:abc123 1 EX 1800
```

9.4 Consultas y DML: 02_consultas_y_dml.redis

Las primeras líneas leen datos (`HGETALL`, `HGET`, `HMGET`, `GET`, `TTL`, `SMEMBERS`). Después se actualiza un precio (`HSET`), se invalida la caché de un producto (`DEL`), se repone stock (`INCRBY`) y se modifica el carrito (`HINCRBY` y `HDEL`).

```
HGETALL producto:1
HGET producto:1 precio
HMGET producto:2 nombre precio
TTL producto:1
GET stock:1
HGETALL carrito:1
TTL carrito:1
SMEMBERS categoria:tecnologia
GET sesion:abc123
TTL sesion:abc123
HSET producto:2 precio 47.00
HGET producto:2 precio
DEL producto:1
EXISTS producto:1
INCRBY stock:3 10
HINCRBY carrito:1 1 1
HDEL carrito:1 3
HGETALL carrito:1
```

9.5 Transacciones: 03_transacciones.redis

`WATCH` vigila el stock del producto 2: si otro cliente lo modifica antes del `EXEC`, la transacción se cancela. Dentro de `MULTI/EXEC` se descuenta una unidad y se suma al ranking como una sola unidad de trabajo.

```
WATCH stock:2
MULTI
DECRBY stock:2 1
ZINCRBY ranking:ventas 1 2
EXEC
```

9.6 Funciones almacenadas: 04_funciones.redis

La biblioteca `tienda` define dos funciones en Lua que Redis ejecuta de forma atómica. comprar devuelve `1` si compró, 0 si no hay stock suficiente y `-1` si el producto no existe. `agregar_carrito` devuelve `1` si agregó el producto y `-1` si el producto no existe, lo que reemplaza a una clave foránea.

```
#!lua name=tienda

local function comprar(keys, args)
  local s = tonumber(redis.call('GET', keys[1]))
  if not s then return -1 end            -- el producto no existe
  if s >= tonumber(args[1]) then
    redis.call('DECRBY', keys[1], args[1])
    redis.call('ZINCRBY', keys[2], args[1], args[2])
    return 1
  end
  return 0                               -- stock insuficiente
end

local function agregar_carrito(keys, args)
  if redis.call('EXISTS', keys[1]) == 0 then return -1 end
  redis.call('HINCRBY', keys[2], args[1], args[2])
  redis.call('EXPIRE', keys[2], 604800)
  return 1
```

```
end

redis.register_function('comprar', comprar)
redis.register_function('agregar_carrito', agregar_carrito)
```

Archivo tal como se ejecuta (la biblioteca va en una sola línea). En FCALL, el 2 es el número de claves; después van las claves y luego los argumentos. Por ejemplo, `FCALL comprar 2 stock:1 ranking:ventas 2 1` compra 2 unidades del producto 1.

```
FUNCTION LOAD REPLACE "#!lua name=tienda\nlocal function comprar(keys, args)\n  local s =
tonumber(redis.call('GET', keys[1]))\n  if not s then return -1 end\n  if s >=
tonumber(args[1]) then\n    redis.call('DECRBY', keys[1], args[1])\n
redis.call('ZINCRBY', keys[2], args[1], args[2])\n    return 1\n  end\n  return
0\nend\nlocal function agregar_carrito(keys, args)\n  if redis.call('EXISTS', keys[1]) ==
0 then return -1 end\n  redis.call('HINCRBY', keys[2], args[1], args[2])\n
redis.call('EXPIRE', keys[2], 604800)\n  return
1\nend\nredis.register_function('comprar',
comprar)\nredis.register_function('agregar_carrito', agregar_carrito)"
FCALL comprar 2 stock:1 ranking:ventas 2 1
FCALL comprar 2 stock:1 ranking:ventas 500 1
FCALL comprar 2 stock:3 ranking:ventas 4 3
FCALL comprar 2 stock:99 ranking:ventas 1 99
FCALL agregar_carrito 2 stock:2 carrito:1 2 1
FCALL agregar_carrito 2 stock:99 carrito:1 99 1
HGETALL carrito:1
```

9.7 Consultas complejas: 05_consultas_complejas.redis

`ZINTERSTORE` cruzamos el ranking de ventas con el Set de una categoría y guarda el resultado, que equivale a un JOIN con filtro: el ranking de ventas solo de los productos de tecnología o de hogar. `WEIGHTS 1 0` conserva el puntaje del ranking. Los resultados

guardados llevan TTL de 60 s porque se pueden recalcular. `ZREVRANK` da la posición de un producto y `ZRANGE ... BYSCORE` filtra por puntaje mínimo.

```
ZREVRANGE ranking:ventas 0 2 WITHSCORES
ZREVRANK ranking:ventas 1
ZRANGE ranking:ventas 2 +inf BYSCORE WITHSCORES
ZINTERSTORE ranking:tecnologia 2 ranking:ventas categoria:tecnologia WEIGHTS 1 0
EXPIRE ranking:tecnologia 60
ZREVRANGE ranking:tecnologia 0 -1 WITHSCORES
ZINTERSTORE ranking:hogar 2 ranking:ventas categoria:hogar WEIGHTS 1 0
EXPIRE ranking:hogar 60
ZREVRANGE ranking:hogar 0 -1 WITHSCORES
GET stock:1
GET stock:2
GET stock:3
```

9.8 Trigger equivalente: notificaciones de expiración

Redis no tiene triggers como un motor relacional, pero puede avisar cuando ocurre un evento sobre una clave. `Con --notify-keyspace-events Ex` (sección 9.1), Redis publica un mensaje cada vez que una clave expira; así se puede reaccionar, por ejemplo, para recalcular la caché de un producto. Demostración con dos terminales.

Terminal 1 (queda escuchando los eventos de expiración):

```
docker exec -it redis-tienda redis-cli subscribe "__keyevent@0__:expired"
```

Terminal 2 (crea una clave que expira en 5 segundos):

```
docker exec -it redis-tienda redis-cli SET prueba:trigger 1 EX 5
```

A los pocos segundos, la terminal 1 muestra un mensaje con el nombre de la clave `prueba:trigger`. Este mecanismo es de tipo publicar/suscribir: si nadie está escuchando en ese momento, el aviso se pierde.

9.9 Prueba de caché en Python: prueba_cache.py

Simula la base de datos con una espera de 200 ms y compara las consultas con y sin caché. Requiere `pip install redis`.

```
import time
import redis

r = redis.Redis(host="localhost", port=6379, decode_responses=True)

# Base de datos simulada (fuente de verdad). Cada consulta tarda 200 ms.
BASE_DE_DATOS = {
    1: {"nombre": "Auriculares inalambricos", "precio": "29.99"},
    2: {"nombre": "Teclado mecanico", "precio": "45.50"},
    3: {"nombre": "Lampara LED", "precio": "12.00"},
}


def consultar_bd(id_producto):
    time.sleep(0.2)
    return BASE_DE_DATOS[id_producto]


def obtener_producto(id_producto, usar_cache=True):
    clave = f"producto:{id_producto}"
    if usar_cache:
        dato = r.hgetall(clave)
        if dato:
            return dato, "HIT"
    dato = consultar_bd(id_producto)
    if usar_cache:
        r.hset(clave, mapping=dato)
        r.expire(clave, 600)
        return dato, "MISS"
    return dato, "SIN CACHE"


def medir_ms(usar_cache):
    inicio = time.perf_counter()
    _, origen = obtener_producto(1, usar_cache)
    return (time.perf_counter() - inicio) * 1000, origen

if __name__ == "__main__":
    r.delete("producto:1")
```

```
    sin_cache = [medir_ms(False)[0] for _ in range(10)]
    promedio_sin = sum(sin_cache) / len(sin_cache)
    print(f"SIN cache: promedio de 10 consultas = {promedio_sin:.1f} ms")

    primera, origen = medir_ms(True)
    print(f"CON cache: primera consulta ({origen}) = {primera:.1f} ms")
    con_cache = [medir_ms(True)[0] for _ in range(9)]
    promedio_con = sum(con_cache) / len(con_cache)
    print(f"CON cache: promedio de las 9 siguientes (HIT) = {promedio_con:.2f} ms")

    print(f"La cache fue {promedio_sin / promedio_con:.0f} veces mas rapida")
    print("TTL restante de producto:1 =", r.ttl("producto:1"), "segundos")
```

9.10 Seguridad

- Este proyecto es una demostración local: Redis corre sin contraseña.
- En producción se activaría requirepass y usuarios con ACL de permisos mínimos, y Redis no se expondría a internet.
- Las claves con TTL (caché, carritos y sesiones) no quedan almacenadas indefinidamente.
- Las credenciales reales irían en variables de entorno y nunca en el repositorio.

9.11 Página web de la tienda: app.py

Aplicación en Flask que se conecta a Redis y muestra la tienda. Requiere `pip install flask redis` y se abre en http://localhost:5001. Aquí se muestra la parte en Python; la plantilla HTML y CSS están en `app.py`, dentro del repositorio.

```
import time

import redis
from flask import Flask, redirect, render_template_string, request, url_for

app = Flask(__name__)
r = redis.Redis(host="localhost", port=6379, decode_responses=True)

USUARIO = 1
```

```
TTL_PRODUCTO = 600

# Base de datos simulada (fuente de verdad). Cada consulta tarda 200 ms.
BASE_DE_DATOS = {
    1: {"nombre": "Auriculares inalambricos", "precio": "29.99"},
    2: {"nombre": "Teclado mecanico", "precio": "45.50"},
    3: {"nombre": "Lampara LED", "precio": "12.00"},
}


def consultar_bd(id_producto):
    time.sleep(0.2)
    return BASE_DE_DATOS[id_producto]


def obtener_producto(id_producto):
    clave = f"producto:{id_producto}"
    inicio = time.perf_counter()
    dato = r.hgetall(clave)
    origen = "HIT"
    if not dato:
        dato = consultar_bd(id_producto)
        r.hset(clave, mapping=dato)
        r.expire(clave, TTL_PRODUCTO)
        origen = "MISS"
    ms = (time.perf_counter() - inicio) * 1000
    return dato, origen, ms


def llamar_funcion(nombre, clave1, clave2, arg1, arg2):
    try:
        return int(r.fcall(nombre, 2, clave1, clave2, arg1, arg2))
    except redis.exceptions.ResponseError:
        return None


def nombre_de(id_producto):
    return BASE_DE_DATOS.get(int(id_producto), {}).get("nombre", f"producto
{id_producto}")


def volver(tipo, texto):
    return redirect(url_for("inicio", tipo=tipo, msg=texto))
```

```
# PAGINA = """..."""  (plantilla HTML y CSS: ver app.py en el repositorio)


ERROR_REDIS = """<!doctype html>
<html lang="es"><head><meta charset="utf-8"><title>Sin conexión</title></head>
<body style="font-family: system-ui, sans-serif; max-width: 560px; margin: 60px auto;
padding: 0 20px;">
<h1>No hay conexión con Redis</h1>
<p>Revisa que Docker Desktop esté abierto y que el contenedor esté corriendo:
<code>docker start redis-tienda</code>. Luego recarga esta página.</p>
</body></html>"""


@app.route("/")
def inicio():
    try:
        productos = []
        for id_producto in BASE_DE_DATOS:
            dato, origen, ms = obtener_producto(id_producto)
            productos.append({
                "id": id_producto,
                "nombre": dato["nombre"],
                "precio": dato["precio"],
                "stock": r.get(f"stock:{id_producto}"),
                "origen": origen,
                "ms": ms,
                "ttl": r.ttl(f"producto:{id_producto}"),
            })
        carrito = [(nombre_de(i), cantidad)
                   for i, cantidad in sorted(r.hgetall(f"carrito:{USUARIO}").items())]
        ranking = [(nombre_de(i), int(puntos))
                   for i, puntos in r.zrevrange("ranking:ventas", 0, 4, withscores=True)]
    except redis.exceptions.ConnectionError:
        return ERROR_REDIS, 503
    return render_template_string(
        PAGINA, productos=productos, carrito=carrito, ranking=ranking, usuario=USUARIO,
        msg=request.args.get("msg"), tipo=request.args.get("tipo", "ok"))


@app.post("/agregar/<int:id_producto>")
def agregar(id_producto):
    resultado = llamar_funcion("agregar_carrito", f"stock:{id_producto}",
f"carrito:{USUARIO}", id_producto, 1)
    if resultado == 1:
        return volver("ok", f"Agregado al carrito: {nombre_de(id_producto)}.")
```

```
    if resultado == -1:
        return volver("error", "Ese producto no existe en el inventario.")
    return volver("error", "Falta cargar los scripts: ejecuta los archivos de la carpeta
scripts/.")

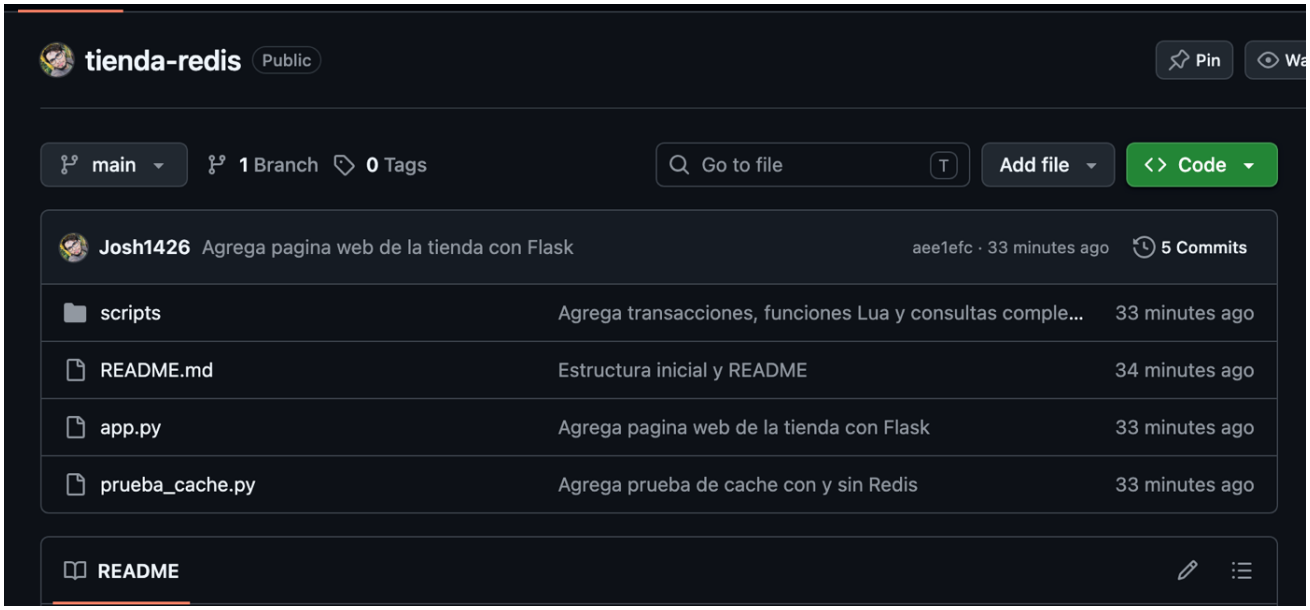

@app.post("/comprar/<int:id_producto>")
def comprar(id_producto):
    resultado = llamar_funcion("comprar", f"stock:{id_producto}", "ranking:ventas", 1,
id_producto)
    if resultado == 1:
        return volver("ok", f"Compra realizada: 1 unidad de {nombre_de(id_producto)}.")
    if resultado == 0:
        return volver("error", f"No hay stock suficiente de {nombre_de(id_producto)}.")
    if resultado == -1:
        return volver("error", "Ese producto no existe en el inventario.")
    return volver("error", "Falta cargar los scripts: ejecuta los archivos de la carpeta
scripts/.")


@app.post("/borrar-cache")
def borrar_cache():
    r.delete(*[f"producto:{i}" for i in BASE_DE_DATOS])
    return volver("ok", "Caché borrada. La próxima lectura irá a la base de datos.")


if __name__ == "__main__":
    app.run(port=5001)
```

10. Publicación en versionadores

Repositorio público: https://github.com/Josh1426/tienda-redis



Enlace al video de exposición (máximo 15 minutos, YouTube o Vimeo):

https://youtu.be/docV2z3Kplg